



# Swagger avec Spring Boot

UP ASI  
Bureau E204

# PLAN DU COURS

- Introduction
- Swagger
- Intégration Swagger
- Configuration Swagger
- Configuration avancée Swagger
- Personnalisation documentation Swagger
- Place à la pratique
- TP

# Introduction

- La **documentation** d'un logiciel aide à expliquer comment utiliser un service ou un programme.
- Elle facilite la maintenance ainsi que le développement ultérieur d'un système.
- C'est une étape très importante dans le cas d'une API (Une API permet de faire communiquer deux programmes ensemble et d'échanger des données).
- Elle contient les informations nécessaires sur les APIs.

# Swagger

- Swagger est une spécification permettant de simplifier la tâche de **documentation des APIs REST**.
- Swagger analyse le code et expose la documentation sur une URL afin d'être utilisé par les développeurs/opérationnels.
- Les API sont toujours synchronisées puisque la documentation des méthodes, des paramètres et des modèles est intégrée au code du serveur.

# Swagger

- Swagger affiche la liste de tous les endpoints de l'API avec des informations détaillées, les paramètres et le schéma de sortie au format JSON.

**Gestion des clients** Client Rest Controller

**POST** /SpringMVC/client/add-client addClient

Parameters Try it out

| Name                                    | Description                                                                                                                                                                                                                                               |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>C</b> * required<br>object<br>(body) | C<br>Example Value   Model<br><pre>{   "categorieClient": "Fidele",   "dateNaissance": "2021-10-31T19:30:11.194Z",   "email": "string",   "idClient": 0,   "nom": "string",   "password": "string",   "prenom": "string",   "profession": "Autre" }</pre> |

Parameter content type  
application/json

Responses Response content type \*/\*

# Swagger

- Swagger-UI fournit une interface stylisée pour tester les endpoints.

← → ↻ ⓘ localhost:8089/SpringMVC/swagger-ui/index.html#/Gestion%20des%20fournisseurss

 **Swagger**  
Supported by SMARTBEAR

Select a definition default

## Swagger Configuration for tpStockProject 1.1.0

[ Base URL: localhost:8089 ]  
<http://localhost:8089/SpringMVC/v2/api-docs>

"Spring Boot Swagger configuration"

**Gestion des clients** Client Rest Controller >

**Gestion des factures** Facture Rest Controller >

**Gestion des fournisseurss** Fournisseur Rest Controller >

**POST** /SpringMVC/fournisseur/add-fournisseur addFournisseur

**PUT** /SpringMVC/fournisseur/assign-fournisseur-to-produit/{fournisseur-id}/{produit-id} assignFournisseurTProduit

# Intégration Swagger

- Pour intégrer Swagger dans une application web, il suffit juste d'ajouter cette dépendance dans le fichier **pom.xml** et puis faire **maven update** de votre projet:

```
<dependency>
```

```
    <groupId>io.springfox</groupId>
```

```
    <artifactId> springfox-boot-starter </artifactId>
```

```
    <version> 3.0.0 </version>
```

```
</dependency>
```

- Le projet **Springfox** automatise la documentation des API construites avec Spring.

# Intégration Swagger

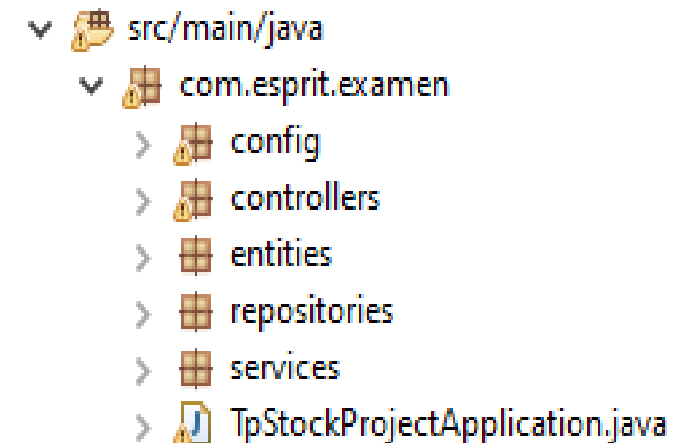
- Pour activer la prise en charge de la spécification **Swagger**, il faut placer l'annotation **@EnableSwagger2** dans la classe main (contenant la méthode main et permettant de scanner les packages de l'application).

```
@EnableSwagger2
```

```
@SpringBootApplication
```

```
public class TpStockProjectApplication {
```

```
    public static void main(String[] args) {  
        SpringApplication.run(TpStockProjectApplication.class, args);  
    }  
}
```





# Configuration Swagger

- Créez une classe **SwaggerConfig** dans un nouveau package appelé **configuration**.

```
@Configuration
public class SwaggerConfig {
    @Bean
    public Docket api() {
        return new Docket(DocumentationType.SWAGGER_2)
            .select()
            .apis(RequestHandlerSelectors.any())
            .paths(PathSelectors.any())
            .build();
    }
}
```

- La classe Docket gère toutes les configurations.

# Configuration Swagger

La classe **Docket** contient les éléments suivants :

- **Select**: Initialiser une classe de ApiSelectorBuilder pour donner l'accès aux méthodes de personnalisation.
- **paths** : filtrer selon l'URI des requêtes.
- **apis** : filtrer la documentation à exposer selon les contrôleurs.
  - basePackage: trier selon le Package.
  - any: documenter toutes les classes dans tous les packages.

# Configuration Swagger

- Redémarrez votre application
- Swagger est capable de générer une documentation détaillée au **format JSON**.

<http://localhost:8089/SpringMVC/v2/api-docs>

```
{
  "swagger": "2.0",
  "info": {
    "description": "Api Documentation",
    "version": "1.0",
    "title": "Api Documentation",
    "termsOfService": "urn:tos",
    "contact": {},
    "license": {
      "name": "Apache 2.0",
      "url": "http://www.apache.org/licenses/LICENSE-2.0"
    }
  },
  "host": "localhost:8089",
  "tags": [
    {
      "name": "client-rest-controller",
      "description": "Client Rest Controller"
    },
    {
      "name": "facture-rest-controller",
      "description": "Facture Rest Controller"
    }
  ]
}
```

# Configuration Swagger

- Swagger est capable d'accéder à une version au **format HTML** via l'adresse suivante:

`localhost:8089/SpringMVC/swagger-ui/index.html`

The screenshot displays the Swagger UI interface. At the top, there's a Swagger logo and a 'Select a definition' dropdown menu set to 'default'. Below this, the 'Api Documentation' section shows the base URL as 'localhost:8089' and provides a link to 'http://localhost:8089/SpringMVC/v2/api-docs'. It also includes links for 'Terms of service' and 'Apache 2.0'. The main content area lists two controllers: 'client-rest-controller' (Client Rest Controller) and 'facture-rest-controller' (Facture Rest Controller). Under 'client-rest-controller', there are six API endpoints listed with their respective HTTP methods and descriptions: POST for adding a client, GET for getting clients by category and date range, PUT for modifying a client, DELETE for removing a client, GET for retrieving all clients, and GET for retrieving a client by ID. The 'facture-rest-controller' section is partially visible at the bottom.

Swagger  
Supported by SMARTBEAR

Select a definition **default**

## Api Documentation <sup>1.0</sup>

[ Base URL: localhost:8089 ]  
<http://localhost:8089/SpringMVC/v2/api-docs>

Api Documentation  
[Terms of service](#)  
[Apache 2.0](#)

### client-rest-controller Client Rest Controller

- POST** /SpringMVC/client/add-client addClient
- GET** /SpringMVC/client/getChiffreAffaireParCategorieClient/{categorieClient}/{startDate}/{endDate} getChiffreAffaireParCategorieClient
- PUT** /SpringMVC/client/modify-client modifyClient
- DELETE** /SpringMVC/client/remove-client/{client-id} removeClient
- GET** /SpringMVC/client/retrieve-all-clients Récupérer la liste des clients
- GET** /SpringMVC/client/retrieve-client/{client-id} Récupérer un client par Id

### facture-rest-controller Facture Rest Controller

# Configuration Swagger

- Informations personnalisées

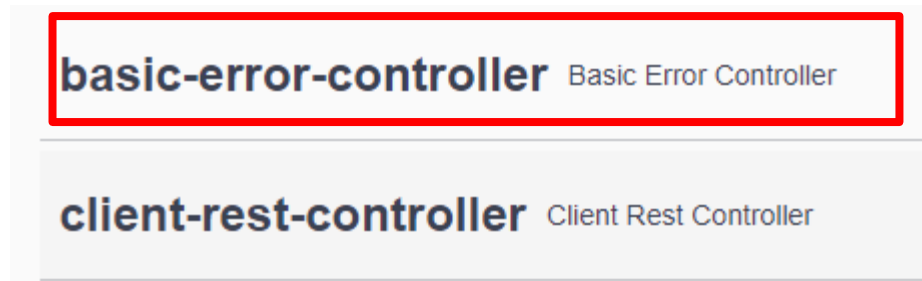


```
@Bean
public Docket api() {
    return new Docket(DocumentationType.SWAGGER_2)
        .select()
        .apis(RequestHandlerSelectors.any())
        .paths(PathSelectors.any())
        .build().apiInfo(apiInfo());
}
private ApiInfo apiInfo() {
    return new ApiInfoBuilder()
        .title("Swagger Configuration for tpStockProject")
        .description("\nSpring Boot Swagger configuration\n")
        .version("1.1.0").build();
}
```

# Configuration avancée Swagger

## Filtrage des APIs

- La documentation contient le contrôleur d'erreur de Spring, pour l'éliminer il faut utiliser **les filtres**.



- La méthode **basePackage** permet de demander à Swagger de ne rien documenter en dehors du package spécifié.

```
.apis(RequestHandlerSelectors.basePackage("com.esprit.spring"))
```

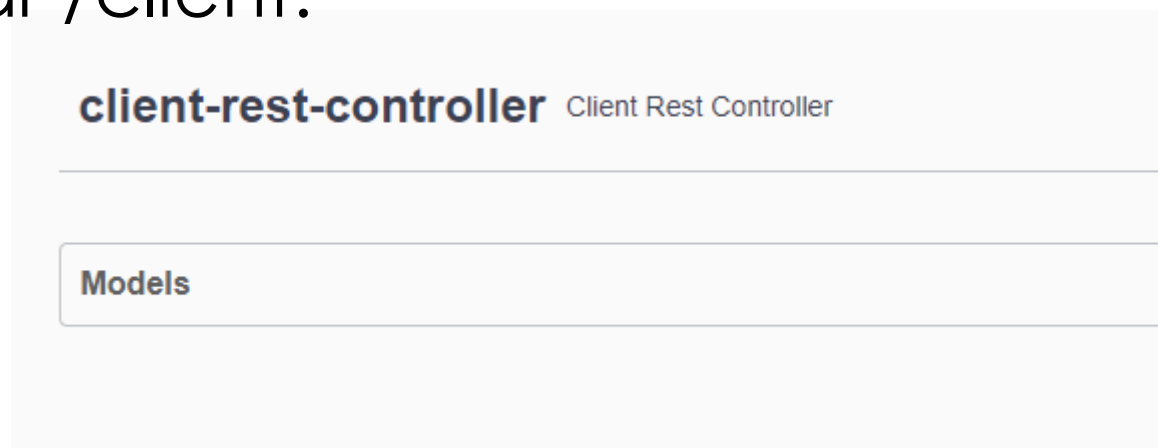
# Configuration avancée Swagger

## Filtrage des APIs

- La méthode **paths** permet d'éliminer une méthode de documentation.

```
.paths(PathSelectors.regex("/client.*"))
```

- PathSelectors.regex("/client.\*")** permet de passer une expression régulière qui n'accepte que les URI commençant par /client.



# Personnalisation documentation Swagger

## Personnaliser la documentation grâce aux annotations

- Pour personnaliser les informations, il faut appliquer des annotations sur les classes des contrôleurs ainsi que leurs méthodes.
- **@Api** : permet d'ajouter une description pour chaque API.
- **@ApiOperation** : permet de définir une description pour chaque opération/méthode.

```
@Api(tags = "Client management")
```

```
@ApiOperation(value = "Récupérer la liste des clients")
```



# Personnalisation documentation Swagger

## Personnaliser la documentation grâce aux annotations

Les informations définies dans ces annotations se retrouvent dans la page `swagger-ui.html`

```
@RestController
@Api(tags = "Gestion des clients")
@RequestMapping("/client")
public class ClientRestController {

    @Autowired
    IClientService clientService;

    @ApiOperation(value = "Récupérer la liste des clients")
    @GetMapping("/retrieve-all-clients")
    @ResponseBody
    public List<Client> getClients() {
        return clientService.retrieveAllClients();
    }
}
```

### Gestion des clients

Client Rest Controller

**POST** /SpringMVC/client/add-client addClient

**GET** /SpringMVC/client/getChiffreAffaireParCategorieClient/{categorieClient}/{startDate}/{endDate} getChiffreAffaireParCategorieClient

**PUT** /SpringMVC/client/modify-client modifyClient

**DELETE** /SpringMVC/client/remove-client/{client-id} removeClient

**GET** /SpringMVC/client/retrieve-all-clients Récupérer la liste des clients

**GET** /SpringMVC/client/retrieve-client/{client-id} Récupérer un client par Id

# Place à la pratique

Une annotation de la couche Controller fournie par Spring va être reflétée dans Swagger.

```
public Client retrieveClient(@PathVariable("client-id") Long clientId) {
```

**@PathVariable** sera reflété comme suit:

The image shows a snippet of a Swagger UI interface. At the top, there is a blue button labeled 'GET' followed by the endpoint path '/SpringMVC/client/retrieve-client/{client-id}'. The path variable '{client-id}' is highlighted with a red rectangle. To the right of the path is the description 'Récupérer un client par Id'. Below the path, there is a section titled 'Parameters'. Under this section, there is a table with two columns: 'Name' and 'Description'. The first row shows 'client-id' with a red asterisk and the word 'required' next to it, and 'client-id' in the description column. Below this, there is a text input field with the value '6'. To the left of the input field, there is text indicating the type and location: 'integer(\$int64)' and '(path)'.

| Name                 | Description |
|----------------------|-------------|
| client-id * required | client-id   |

integer(\$int64)  
(path)

# Place à la pratique

## ❖ Ajouter un client

Gestion des clients Client Rest Controller

POST /client/add-client addClient

Parameters

Try it out

| Name                                    | Description             |
|-----------------------------------------|-------------------------|
| <b>c</b> * required<br>object<br>(body) | C<br>Edit Value   Model |

```
{  "categorieClient": "Fidele",  "dateNaissance": "1995-01-02",  "email": "amina.sayed@esprit.tn",  "nom": "sayedi",  "password": "123",  "prenom": "amina",  "profession": "Docteur"}
```

Cancel

Parameter content type

application/json

Execute

Code

Details

200

Response body

```
{  "idClient": 7,  "nom": "sayedi",  "prenom": "amina",  "dateNaissance": "1995-01-02T00:00:00.000+00:00",  "email": "amina.sayed@esprit.tn",  "password": "123",  "profession": "Docteur",  "categorieClient": "Fidele"}
```

# Place à la pratique

**POST** /SpringMVC/produit/add-produit/{idRayon}/{idStock} addProduit

Parameters Try it out

| Name                                                    | Description                                                                                                                                                                                                                                                                                                                         |
|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>idRayon</b> * required<br>integer(\$int64)<br>(path) | idRayon<br><div>idRayon - idRayon</div>                                                                                                                                                                                                                                                                                             |
| <b>idStock</b> * required<br>integer(\$int64)<br>(path) | idStock<br><div>idStock - idStock</div>                                                                                                                                                                                                                                                                                             |
| <b>p</b> * required<br>object<br>(body)                 | p<br>Example Value   Model<br><pre>{   "code": "string",   "detailProduit": {     "categorieProduit": "Alimentaire",     "dateCreation": "2021-10-31T20:04:57.709Z",     "dateDerniereModification": "2021-10-31T20:04:57.709Z",     "idDetailProduit": 0   },   "idProduit": 0,   "libelle": "string",   "prixUnitaire": 0 }</pre> |

Parameter content type  
application/json

# TP

Exposer les services implémentés avec Swagger pour les tester.

# Swagger avec spring boot

Si vous avez des questions, n'hésitez pas à nous contacter :

**Département Informatique**  
**UP ASI**

Bureau E204